VECTOR

MICROSAR I-PDU Multiplexer

Technical Reference

Version 3.01.00

Authors	Safiulla Shakir, Markus Bart, Gunnar Meiss
Status	Released

Document Information

History

Author	Date	Version	Remarks
Safiulla Shakir	2011-12-06	1.00.00	Initial CFG5 version derived from TechnicalReference_ASR_IpduM.pdf
Markus Bart	2012-08-23	2.00.00	ESCAN00058313 AR4-160: Support AUTOSAR 4.0.3
Markus Bart	2013-01-29	2.01.00	ESCAN00063294 AR4-197: Support BIG_ENDIAN Copy Segments in IpduM
Gunnar Meiss	2013-04-04	2.02.00	ESCAN00064368 AR4-325: Post-Build Loadable
Markus Bart	2014-11-05	2.03.00	AR4-698: Post-Build Selectable (Identity Manager)
Markus Bart	2014-12-01	2.04.00	FEAT-229: Support 16bit selector in IpduM [AR4-927]
Markus Bart	2015-08-11	2.05.00	FEAT-1315: IPDUM for CAN-FD supporting nPdu2Frame-Mapping
Gunnar Meiss	2016-02-25	2.06.00	FEAT-1631: Trigger Transmit API with SduLength In/Out according to ASR4.2.2
Markus Bart	2017-12-21	2.07.00	STORYC-3561: made CalculateSizeOfContainer an external API
Markus Bart	2018-03-06	2.08.00	FEAT-2072: Introduce separate main functions for reception and transmission for IPDUM
Markus Bart	2018-05-04	3.00.00	STORYC-5229: Add Container PDU Configuration to Doc_TechRef
Markus Bart	2019-01-18	3.01.00	STORYC-6469: Improve documentation and validation for Container PDUs and FlexRay

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_IPDUMultiplexer.pdf	2.2.0
[2]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	1.6.0
[3]	Vector	TechnicalReference_PostBuildLoadable.pdf	1.0.0

© 2019 Vector Informatik GmbH



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.



Caution
This symbol calls your attention to warnings.

© 2019 Vector Informatik GmbH

Contents

1 Component History 7

2 Introduction 8
2.1 Architecture Overview 9

3 Functional Description 11
3.1 Features 11
3.2 Initialization 12
3.3 States 12
3.4 Main Functions 12
3.5 Error Handling 13
3.5.1 Development Error Reporting 13
3.6 nPdu-to-Frame mapping for CAN-FD 13

4 Integration 14
4.1 Scope of Delivery 14
4.1.1 Static Files 14
4.1.2 Dynamic Files 14
4.2 Critical Sections 15

5 Configuration 16
5.1 Container PDUs 16
5.1.1 Meaning of “triggering” a Container PDU 16
5.1.2 Data provision: Direct vs. Trigger Transmit 16
5.1.3 Triggers and their effect on timing behavior 16
5.1.4 DET error 49 (IPDUM_E_QUEUEOVFL) 17
5.1.5 Timing behavior with high bus load 17
5.1.6 SecOC PDUs in IpduM Container PDUs 18
5.1.7 Understanding Tx PDU collection semantics “last is best” 18
5.2 Multiplex PDUs 18
5.2.1 Triggering a transmission 18
5.2.2 Just-in-time update (JIT update) 19
5.2.2.1 Triggered part with update bit 19
5.2.2.2 Non-triggered part with update bit 19
5.2.2.3 Both parts with update bit 19
5.2.2.4 Using JIT update without update bits 19

6 API Description 20
6.1 Services provided by IPDUM 20

6.1.1 IpduM_InitMemory 20
6.1.2 IpduM_Init 20
6.1.3 IpduM_Transmit 21
6.1.4 IpduM_MainFunction 22
6.1.5 IpduM_GetVersionInfo 22
6.1.6 IpduM_CalculateSizeOfContainer 23
6.2 Services used by IPDUM 24
6.3 Callback Functions 25
6.3.1 IpduM_RxIndication 25
6.3.2 IpduM_TxConfirmation 25
6.3.3 IpduM_TriggerTransmit 26

7 Configuration 27
7.1 Configuration of Post-Build 27

8 AUTOSAR Standard Compliance 28
8.1 Deviations 28
8.2 Additions/ Extensions 28
8.3 Limitations 28

9 Glossary and Abbreviations 29
9.1 Glossary 29
9.2 Abbreviations 30

10 Contact 31

Illustrations

Figure 2-1 AUTOSAR 4.1 Architecture Overview 9
Figure 2-2 AUTOSAR architecture 9
Figure 2-3 Interfaces to adjacent modules of the IPDUM 10

Tables

Table 1-1 Component history 7
Table 3-1 Supported AUTOSAR standard conform features 11
Table 3-2 Not supported AUTOSAR standard conform features 12
Table 3-3 Features provided beyond the AUTOSAR standard 12
Table 4-1 Static files 14
Table 4-2 Generated files 15
Table 6-1 IpduM_InitMemory 20
Table 6-2 IpduM_Init 21
Table 6-3 IpduM_Transmit 21
Table 6-4 IpduM_MainFunction 22
Table 6-5 IpduM_GetVersionInfo 22
Table 6-6 IpduM_CalculateSizeOfContainer 23
Table 6-7 Services used by the IPDUM 24
Table 6-8 IpduM_RxIndication 25
Table 6-9 IpduM_TxConfirmation 26
Table 6-10 IpduM_TriggerTransmit 26
Table 9-1 Glossary 30
Table 9-2 Abbreviations 30

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00	▶ Component in conformance with AUTOSAR 3.2.1
2.00	▶ AUTOSAR 4.0.3
2.01	▶ BIG_ENDIAN copy segments
2.02	▶ Support post-build loadable
3.00	▶ Support post-build selectable ▶ Support deleting container at post-build time
4.00	▶ Extend support for module initialization
6.00	▶ Support 16 bit selector ▶ Support selector using more than one byte (i.e. crossing byte boundaries)
6.01	▶ Support nPdu-to-Frame Mapping for CAN-FD

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module IPDUM as specified in [1].

Supported AUTOSAR Release*:	4.x	
Supported Configuration Variants:	PRE-COMPILE [SELECTABLE] POST-BUILD-LOADABLE [SELECTABLE]	
Vendor ID:	IPDUM_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	IPDUM_MODULE_ID	52 decimal (according to ref. [2])

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

Multiplexing is a concept generally used to save CAN identifiers where an I-PDU with a CAN ID is used to carry different I-PDUs thereby saving the CAN IDs for the I-PDUs it carries. The IpduM can be used also with Flexray and LIN.

I-PDU multiplexing means using the same PCI of a PDU with more than one unique layouts of its SDU. The SDU in a multiplexed I-PDU contains a selector field which is used to determine the layout of the SDU being multiplexed or de-multiplexed.
The AUTOSAR IpduM multiplexes a dynamic part along with or without a static part.
Each dynamic and static part is an independent signal-I-PDU in AUTOSAR BSW module Com.

2.1 Architecture Overview

The following figure shows where the IPDUM is located in the AUTOSAR architecture.

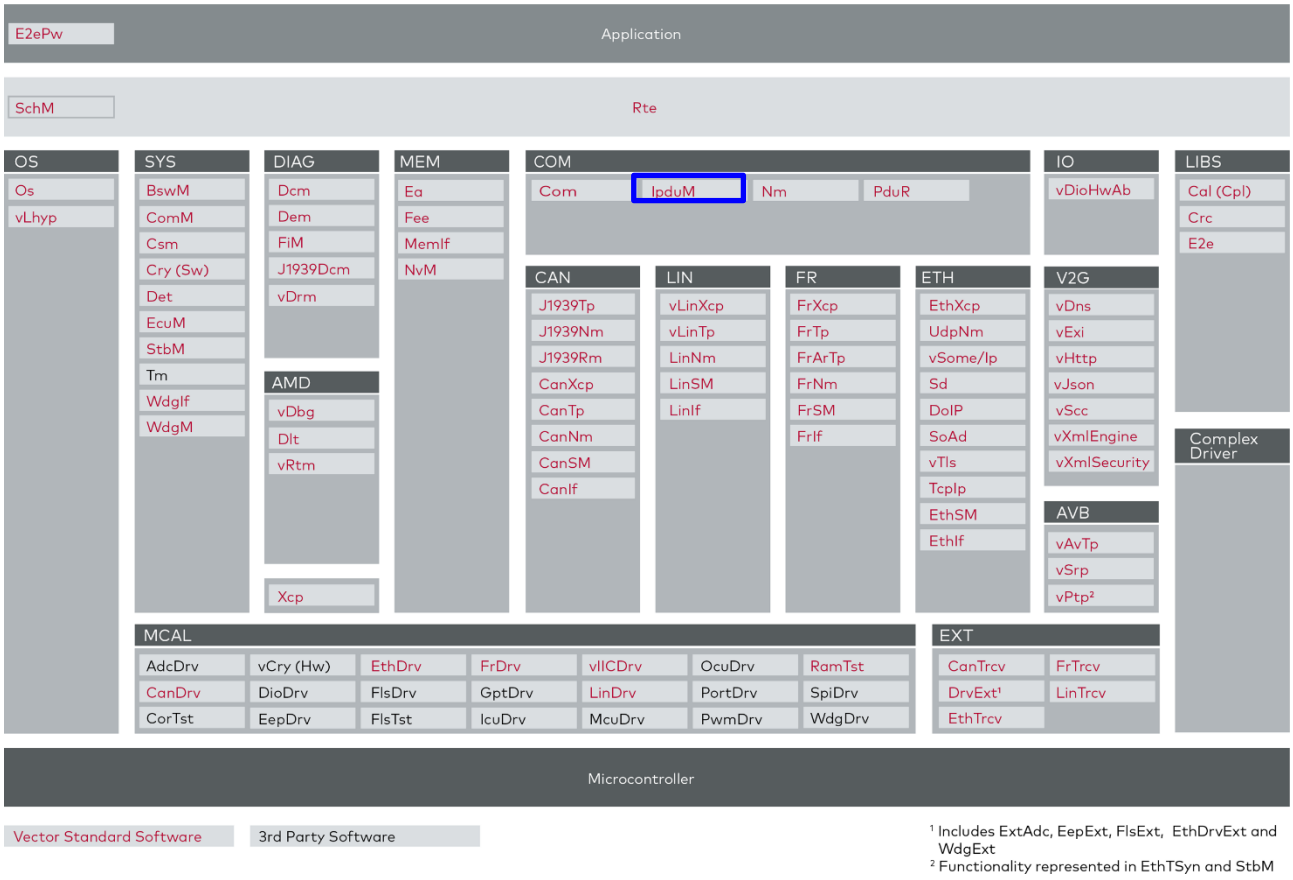


Figure 2-1 AUTOSAR 4.1 Architecture Overview

Figure 2-2 AUTOSAR architecture

The next figure shows the interfaces to adjacent modules of the IPDUM. These interfaces are described in chapter 4.2.

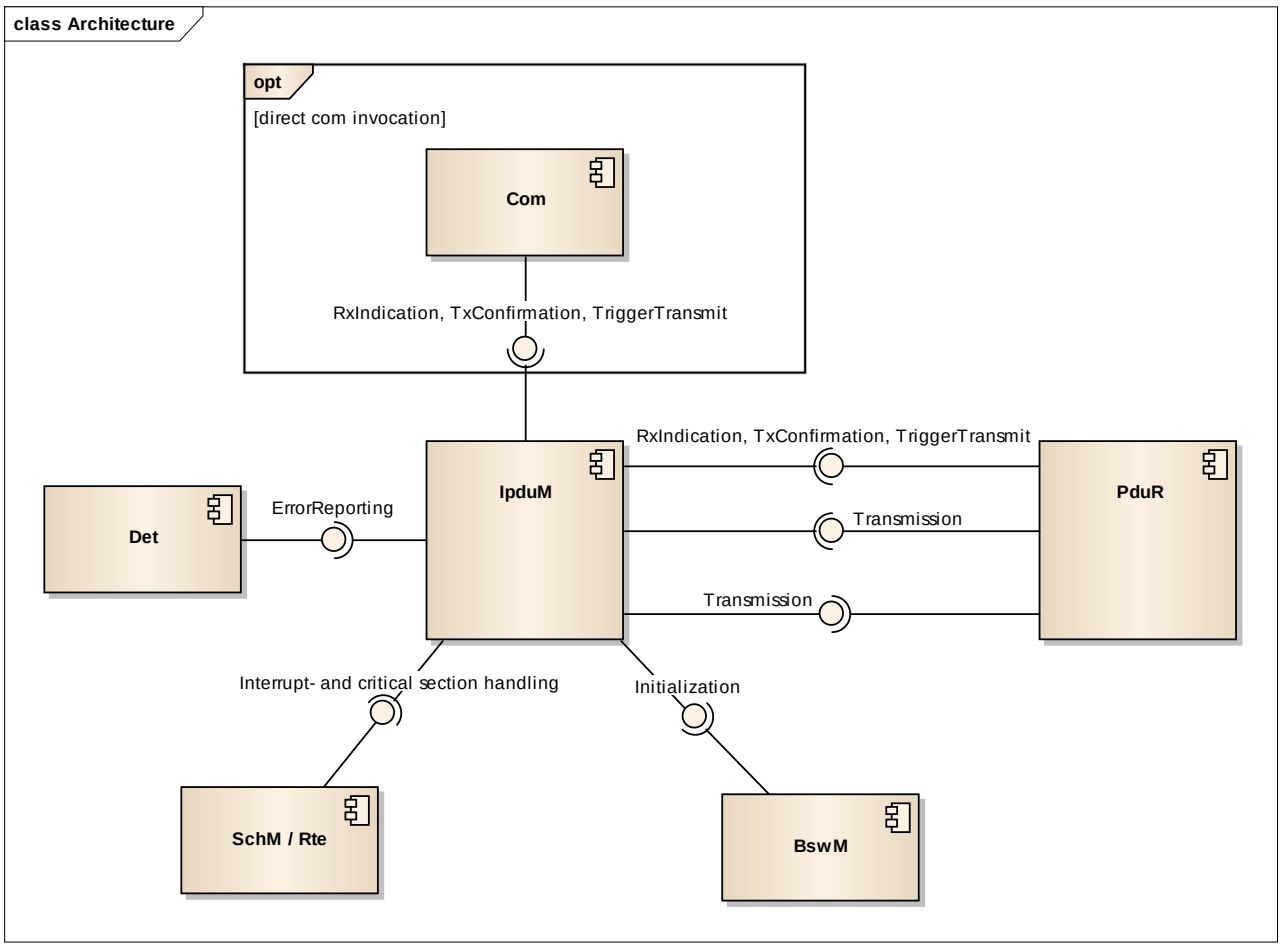


Figure 2-3 Interfaces to adjacent modules of the IPDUM

Normally the IpduM only interacts with the PduR. If required the IpduM can interface Com directly bypassing the PduR while indicating receptions and confirming transmissions.

The IpduM also uses the interfaces to the BSW Scheduler and Det.

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the IPDUM.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

- Table 3-1 Supported AUTOSAR standard conform features
- Table 3-2 Not supported AUTOSAR standard conform features
- Table 3-3 Features provided beyond the AUTOSAR standard

For further information of not supported features see also chapter 7.

The following features specified in [1] are supported:

Supported Feature
Configuration
> Multiplexing of the static and dynamic parts.
> Event base transmission triggering of the multiplexed I-PDU(s).
> Forwarding confirmations of all transmitted multiplexed I-PDUs to the BSW module AUTOSAR Com.
> Timeout handling of transmission confirmations.
> De-multiplexing and indicating receptions of the de-multiplexed dynamic parts and static part to the BSW module AUTOSAR Com.
> Static part configuration support in multiplex I-PDU(s).
> AUTOSAR EcuC format
> Initialization of transmission multiplex I-PDU buffers. This is required by e.g. FrIf in case the transmission of a multiplex I-PDU is triggered by interface layer BSW module before the transmission of their mapped dynamic and or static parts is requested.
> Just-in-Time Update of dynamic and / or static parts.
> DET error reporting.

Table 3-1 Supported AUTOSAR standard conform features

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
> Configuring of transmission confirmations of transmitted multiplexed I-PDUs AUTOSAR Com.

Table 3-2 Not supported AUTOSAR standard conform features

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
> nPdu-to-Frame mapping for CAN-FD

Table 3-3 Features provided beyond the AUTOSAR standard

3.2 Initialization

The IpduM is initialized by calling the API `IpduM_Init()`. This is done by the AUTOSAR BSW module EcuM. If this is not the case, an adequate and suitable solution has to be provided.

The API initializes all the transmission multiplex I-PDU(s) buffer(s) and all runtime relevant module variables. The successful execution of the API initializes the IpduM.



Caution
No IpduM function should pre-empt `IpduM_Init()`. The function should be called on task level and should not to be interrupted by other administrative function calls.

3.3 States

The IpduM has the states: uninitialized and initialized. By calling IpduM_Init(), the state is changed from uninitialized to initialized.

3.4 Main Functions

IpduM provides `IpduM_MainFunctionTx()` and `IpduM_MainFunctionRx()`. Both should be called cyclically by the Basic Software Scheduler or a similar component.

The `IpduM_MainFunctionTx()` takes care of timeout handling of transmission confirmations for multiplex and container PDUs, send timeout handling of container PDUs, transmission of queued container PDUs.

The `IpduM_MainFunctionRx()` takes care of reception of queued container PDUs.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1], if development error reporting is enabled (i.e. pre-compile parameter `IPDUM_DEV_ERROR_DETECT==STD_ON`). The DET reporting can be enabled during pre-compile time.

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported IPDUM ID is 52.

The reported services IDs identify the IPDUM services as specified by [1]. For the service IDs and their related service names see IpduM.h Doxygen DefGroup “IpduMApiIds”.

The errors reported to DET are described in section 7.6 of [1].

3.6 nPdu-to-Frame mapping for CAN-FD

This feature was backported from AUTOSAR 4.2.1. It can be used to collect multiple PDUs and transfer them in one CAN-FD frame, for saving bandwidth or tunneling several classic CAN busses over one CAN-FD bus. The IpduM module handles packing and unpacking of those frames similar to the multiplexing feature.

Several “contained” PDUs can be put into a single “container” PDU in arbitrary order and unpacked in the same order on the receiver side. Transmission is triggered by one of these conditions:

- Contained PDU send timeout runs out.
- Container PDU send timeout runs out.
- Size threshold of container PDU is reached.
- Contained PDU to be added does not fit into the container PDU anymore.
- Contained PDU is configured for immediate transmission.
- Container PDU is configured for immediate transmission.

On the receiver side, the contained PDUs are retrieved from the container PDU again and RxIndications are created for every single contained PDU in the same order they were put into the container PDU. For even greater flexibility, a container PDU on the receiver side can be configured to accept any contained PDU instead of just the ones that are configured.

4 Integration

This chapter provides information necessary for the integration of the MICROSAR IPDUM into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the IPDUM contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Source Code Delivery	Object Code Delivery	Description
IpduM.c	■		This is the source file of the IPDUM
IpduM.h	■	■	This is the header file of IPDUM
IpduM_Cbk.h	■	■	This is the callback header file of IPDUM

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator.

File Name	Description
IpduM_Cfg.h	This file contains: > global constant macros > global function macros > global data types and structures > global data prototypes > global function prototypes of CONFIG-CLASS PRE-COMPILE data.
IpduM_Lcfg.h	This file contains: > global constant macros > global function macros > global data types and structures > global data prototypes > global function prototypes of CONFIG-CLASS LINK data.
IpduM_Lcfg.c	This file contains: > local constant macros > local function macros > local data types and structures

File Name	Description
	> local data prototypes > local data > global data of CONFIG-CLASS LINK and PRE-COMPILE data.
IpduM_PBcfg.h	This file contains: > global constant macros > global function macros > global data types and structures > global data prototypes > global function prototypes of CONFIG-CLASS POST-BUILD data.
IpduM_PBcfg.c	This file contains: > local constant macros > local function macros > local data types and structures > local data prototypes > local data > global data of CONFIG-CLASS POST-BUILD data.

Table 4-2 Generated files

4.2 Critical Sections

The critical section IPDUM_EXCLUSIVE_AREA_0 has to lock global interrupts to protect common critical sections.

5 Configuration

This chapter provides information for configuration of common use-cases of the MICROSAR IPDUM.

5.1 Container PDUs

5.1.1 Meaning of “triggering” a Container PDU

When a Container PDU is triggered, it is closed and cannot be modified anymore. The IpduM first attempts to transmit it right away, and if this fails, the Container PDU is sent to the queue for later transmission.

If the data provision is set to Trigger Transmit, triggering a Container PDU always sends it to the queue but additionally sends a transmit request to the lower layer to inform it of new data ready to be fetched.

5.1.2 Data provision: Direct vs. Trigger Transmit

The parameter IpduMContainerTxTriggerMode is used to determine when to remove a Container PDU from the IpduM’s internal buffer.

- IPDUM_DIRECT: the PDU is removed on triggering
- IPDUM_TRIGGERTRANSMIT: the PDU is removed when it is fetched via a call to IpduM_TriggerTransmit

So the value should be IPDUM_DIRECT for every bus that does not fetch data via Trigger Transmit.

5.1.3 Triggers and their effect on timing behavior

There are several ways of triggering a Container PDU:

- Contained PDU Send Timeout
- Container PDU Send Timeout
- Size Threshold
- Contained PDU Trigger Always
- Container PDU First Contained PDU Trigger
- Overfilling a Container PDU, i.e. trying to add more PDUs than what fits

All send timeout triggers are evaluated in the next main function cycle, except for a send timeout value of 0. All other triggers including send timeouts with a send timeout value of 0 are evaluated in the context of the function IpduM_Transmit.

The Container PDU First Contained PDU Trigger does not close the Container PDU and just calls PduR_IpduMTransmit. This way, a Trigger Transmit bus is notified of available data but more PDUs can still be added to the Container PDU until the actual IpduM_TriggerTransmit call is made.

The Overfilling trigger is a non-configurable trigger and depends on runtime behavior. Whenever a PDU is about to be added to a Container PDU but does not fit anymore, the Container PDU is triggered immediately. This can lead to unplanned transmissions and usually occurs if too many PDUs are sent to the IpduM at a time.

5.1.4 DET error 49 (IPDUM_E_QUEUEOVFL)

Do enable DET reporting for IpduM during development if you are using Container PDUs. If you get DET error 49 (IPDUM_E_QUEUEOVFL), you are losing PDUs due to queue overflows. Usually, increasing the queue size does not solve this problem, it just makes the time until the problem occurs longer. There are two common reasons for queue overflows in IpduM.

If the amount of PDUs configured to be transmitted in a Container PDU exceeds the amount of Container PDUs the bus can accept, either because of bus load or because of a fixed transmission schedule (e.g. on FlexRay), the queue will overflow, no matter how large the queue size.

If the timings of the PDUs and the according Container PDU are carefully matched for a near-100% filled Container PDU every time and still, queue overflows occur, it is usually because of hardware timer jitters or late tx confirmations or events that temporarily block transmission of a Container PDU, like a bus-off or a negative response to a transmit call. The paragraph Timing behavior with high bus load discusses this case in more detail.

5.1.5 Timing behavior with high bus load

If every available time-slot of the bus must be used (in a time-triggered bus like FlexRay) or if the general bus load is near 100% (in any other bus), the contents of the Container PDUs must be well planned. A single over-filled Container PDU can lead to a chain of PDUs overflowing into the next Container PDU, and mess up the timing.

First, it is planned which PDUs are going to be in which transmitted instance of a given Container PDU. Next, it is planned in which order they are put into the Container PDU. This ensures the trigger by Container overflow is predictable if it does occur.

Finally, it must be planned how the Container PDU is going to be triggered exactly when all the planned PDUs are in the Container PDU. In a scenario where every Container PDU is nearly full, a size threshold on the Container PDU or a “trigger always” trigger on the last PDU being added are good choices. The overfilling trigger should be avoided.

One thing to keep an eye on are cycle times and offsets of periodically transmitted PDUs. They often stack up. If you e.g. have a PDU that is transmitted every 40ms and another one that is transmitted every 100ms, they will be triggered at the same time every 200ms. This is where an overfilling trigger can occur. By using offsets, this problem can be solved, in this case an offset of 10ms would work while an offset of 20ms would not.

Finally, always keep a periodic time-slot with enough free space in a Container PDU to catch up on overfilling.

5.1.6 SecOC PDUs in IpduM Container PDUs

If the IpduM has Container Tx PDUs that contain SecOC PDUs, the collection semantics “last is best” must not be used. The current IpduM Implementation issues two trigger transmit calls to the upper layer for “last is best” PDUs. The SecOC however requires a re-authentication between consecutive trigger transmit calls to the same PDU, which usually does not happen fast enough because it depends on the SecOC main function being called between those two trigger transmit calls.

In summary: if “last is best” is used here, the SecOC will return a negative result for the second trigger transmit call and the IpduM will delete the PDU from the Container Tx PDU. Use the “queued” semantics.

5.1.7 Understanding Tx PDU collection semantics “last is best”

The parameter IpduMContainedTxPduCollectionSemantics has two values – “queued” and “last is best”. The value “queued” is straightforward: the PDUs are added to the Container PDU in the order they are transmitted to the IpduM, allowing duplicates, and in the receiving ECU they are unpacked and forwarded to the upper layer in the exact same order, including the duplicates.

The value “last is best” does two things. It avoids duplicates by using the last version of the same PDU, and it updates the PDU with new data right before transmission, using trigger transmit.

Using the “last is best” collection semantics is useful in these cases:

- Some PDUs have new data more often than the according Container PDU is transmitted, so the Container PDU might transmit old data.
- Receiving the same PDU twice in a very short time poses a problem on the receiving ECU.

The “last is best” collection semantics should not be used in these cases:

- All transmitted PDUs must be received (for example if a counter value is used for detecting missing messages for security reasons).
- The Container PDU is triggered before data in its PDUs can get old, requiring an update.
- The Container PDU is triggered fast enough in relation to the PDUs, so duplicate PDUs in the same container are not to be expected.

Configuring a PDU “last is best” will make the ECU slower. It adds at least two function calls to every contained PDU for every transmission. Only use it when it is necessary.

5.2 Multiplex PDUs

5.2.1 Triggering a transmission

The trigger mode can be set to static part, dynamic part, both or none. This means the multiplexed PDU is forwarded to the bus if a static part PDU is received, a dynamic part

PDU is received, any of them is received or never. The latter is useful on a trigger transmit bus like FlexRay.

5.2.2 Just-in-time update (JIT update)

The JIT update was introduced to ensure the update bits from COM module will appear on the bus. This is necessary because of IpduM's internal buffer acting in a last-is-best manner.

5.2.2.1 Triggered part with update bit

If the (static/dynamic) part that triggers the multiplex PDU transmission has an update bit configured in COM module, it should be configured to clear the update bit on transmit. JIT update is not necessary here.

5.2.2.2 Non-triggered part with update bit

If the (static/dynamic) part, that does not trigger the multiplex PDU transmission, has an update bit configured in COM module, it should be configured to clear the update bit on trigger transmit and to use JIT update for this (static/dynamic) part.

5.2.2.3 Both parts with update bit

If both the static and dynamic parts have update bits configured in COM module, the part that triggers multiplex PDU transmission should be configured to clear the update bit on transmit, and the non-triggering part on trigger transmit with JIT update.

5.2.2.4 Using JIT update without update bits

There are valid use cases where JIT update is used without update bits. This can be useful for getting the most recent available data for the non-triggering part without the need for fast cyclic transmission.

In this case, configure JIT update for the non-triggering part.

6 API Description

For an interfaces overview please see Figure 2-3.

6.1 Services provided by IPDUM

6.1.1 IpduM_InitMemory

Prototype	
`void IpduM_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
The function initializes variables, which cannot be initialized with the startup code.	
Particularities and Limitations	
> IpduM_Init() is not called yet. > The function is called by the Application.	
Expected Caller Context	
> The function must be called on task level.	

Table 6-1 IpduM_InitMemory

6.1.2 IpduM_Init

Prototype	
`void IpduM_Init (const IpduM_ConfigType* config)`	
Parameter	
`config`	> NULL_PTR in the IPDUM_CONFIGURATION_VARIANT_PRECOMPILE > Pointer to the IpduM configuration data in the IPDUM_CONFIGURATION_VARIANT_POSTBUILD_LOADABLE
Return code	
void	none
Functional Description	
This function initializes the IpduM and performs configuration consistency checks. If the initialization is performed successfully the IpduM is initialized.	
Particularities and Limitations	
> IpduM_InitMemory() has been executed, if the startup code does not initialize variables. The IpduM is in the state uninitialized. > No IpduM function shall not pre-empt IpduM_Init()	
Expected Caller Context	

> The function is used by the Ecu State Manager.

Table 6-2 IpduM_Init

6.1.3 IpduM_Transmit

Prototype	
`StdReturnType` **`IpduM_Transmit`** `(PduIdType PdumTxPduId, const PduInfoType* PduInfoPtr)`	
Parameter	
`PdumTxPduId`	ID of transmit request (static or dynamic part). Identifies the payload.
`PduInfoPtr`	Payload information of the I-PDU (pointer to data and data length).
Return code	
`StdReturnType`	E_OK: The request was accepted by the IpduM and by the destination layer. E_NOT_OK: The request was rejected because: the IpduM is not initialized or the provided PduInfoPtr is NULL or the SduDataPtr of the PduInfoPtr is NULL or the PdumTxPduId is not in the expected range or the destination layer did not accept the transmission request.
Functional Description	
The function serves to request the transmission of a static or dynamic part.	
Particularities and Limitations	
> The IpduM_Init() has been executed successfully. > The function is non-reentrant for the same PdumTxPduId.	
Expected Caller Context	
> The function is called by the PduR. > The function is called in an interrupt context and at the task level	

Table 6-3 IpduM_Transmit

6.1.4 IpduM_MainFunction

Prototype	
`void IpduM_MainFunction (void)`	
Parameter	
`void`	none
Return code	
`void`	none
Functional Description	
This function shall perform the processing of the IpduM processing which is not directly initiated by calls from the PduR. A call to IpduM_MainFunction returns simply if IpduM was not previously initialized with a call to IpduM_Init.	
Particularities and Limitations	
> The IpduM_Init() has been executed successfully. > The function is called cyclically by the BSW scheduler	
Expected Caller Context	
> The function is called on task level	

Table 6-4 IpduM_MainFunction

6.1.5 IpduM_GetVersionInfo

Prototype	
`void IpduM_GetVersionInfo (Std_VersionInfoType* versioninfo)`	
Parameter	
`versioninfo`	Pointer to the structure to write the version info to.
Return code	
`void`	none
Functional Description	
Returns the version information of the IpduM.	
Particularities and Limitations	
> none	
Expected Caller Context	
> The function can be called on interrupt and task level. > The function is called by the Application.	

Table 6-5 IpduM_GetVersionInfo

6.1.6 IpduM_CalculateSizeOfContainer

Prototype	
`uint32 IpduM_CalculateSizeOfContainer (PduLengthType headerSize, const PduInfoType* PduInfoPtr)`	
Parameter	
`headerSize`	Size of the header.
`PduInfoPtr`	Payload information of the I-PDU (pointer to data and data length).
Return code	
`uint32`	Size of the container PDU.
Functional Description	
Returns the size of the container PDU given in PduInfoPtr.	
Particularities and Limitations	
> none	
Expected Caller Context	
> The function can be called on interrupt and task level. > The function is called by Microsar SecOC.	

Table 6-6 IpduM_CalculateSizeOfContainer



Caution
The function IpduM_CalculateSizeOfContainer may only be called by Microsar SecOC, see safety manual.

6.2 Services used by IPDUM

In the following table services provided by other components, which are used by the IPDUM are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
Det	Det_ReportError
PduR	PduR_IpduMTransmit
PduR	PduR_IpduMTxConfirmation
PduR	PduR_IpduMRxIndication
PduR	PduR_IpduMTriggerTransmit
Com	Com_RxIndication
Com	Com_TxConfirmation
Com	Com_TriggerTransmit
BSW Scheduler	SchM_Enter_IpduM
BSW Scheduler	SchM_Exit_IpduM

Table 6-7 Services used by the IPDUM

6.3 Callback Functions

This chapter describes the callback functions that are implemented by the IPDUM and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `IpduM_Cbk.h` by the IPDUM.

6.3.1 IpduM_RxIndication

Prototype	
`void IpduM_RxIndication (PduIdType RxPduId, const PduInfoType* PduInfoPtr)`	
Parameter	
`RxPduId`	ID of received multiplexed I-PDU. Identifies the payload.
`PduInfoPtr`	Contains the length (SduLength) of the received I-PDU and a pointer to a buffer (SduDataPtr) containing the I-PDU.
Return code	
void	none
Functional Description	
This service indicates the complete reception of an IpduM I-PDUs. It de-multiplexes the received multiplexed I-PDU.	
Particularities and Limitations	
> The IpduM_Init() has been executed successfully > The function is non-reentrant for the same PdumRxPduId.	
Expected Caller Context	
> The function is called by the PduR. The function is called in an interrupt context or at the task level	

Table 6-8 IpduM_RxIndication

6.3.2 IpduM_TxConfirmation

Prototype	
`void IpduM_TxConfirmation (PduIdType TxPduId)`	
Parameter	
`TxPduId`	ID of transmitted multiplexed IPDU.
Return code	
`void`	none
Functional Description	
This service confirms the transmissions of the dynamic and static parts contained in the multiplexed IPDU whose transmission is confirmed.	
Particularities and Limitations	
> The IpduM_Init() has been executed successfully > The function is non-reentrant for the same PdumTxPduId.	

Expected Caller Context
> The function is called by the PduR. The function is called in an interrupt context or at the task level

Table 6-9 IpduM_TxConfirmation

6.3.3 IpduM_TriggerTransmit

Prototype	
`Std_ReturnType` **`IpduM_TriggerTransmit`** `(PduIdType TxPduId, PduInfoType* PduInfoPtr)`	
Parameter	
`TxPduId`	ID of transmission multiplexed IPDU.
`PduInfoPtr`	Contains a pointer to a buffer (SduDataPtr) to where the SDU data shall be copied, and the available buffer size in SduLengh. On return, the service will indicate the length of the copied SDU data in SduLength.
Return code	
`Std_ReturnType`	E_OK: SDU has been copied and SduLength indicates the number of copied bytes. E_NOT_OK: No data has been copied, because IpduM is not initialized or TxPduId is not valid or PduInfoPtr is NULL_PTR or SduDataPtr is NULL_PTR or SduLength is to small or the SDU data could not be updated because the just in time update via a trigger transmit call of the upper layer was not successful for multiplexed messages.
Functional Description	
This service copies the transmission multiplexed IPDU to the provided buffer.	
Particularities and Limitations	
> The IpduM_Init() has been executed successfully > The function is non-reentrant for the same TxPduId	
Expected Caller Context	
> The function is called by the PduR module. The function is called in an interrupt context or at the task level	

Table 6-10 IpduM_TriggerTransmit

7 Configuration

The IPDUM in the EcuC can be configured through the Vector configuration and generation tool CFG5.

7.1 Configuration of Post-Build

The configuration of post-build loadable is described in [3].

8 AUTOSAR Standard Compliance

8.1 Deviations

No deviations.

8.2 Additions/ Extensions

No additions/ extensions.

8.3 Limitations

See Table 3-2 Not supported AUTOSAR standard conform features.

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
BSWMD	The BSWMD is a formal notation of all information belonging to a certain BSW artifact (BSW module or BSW cluster) in addition to the implementation of that artifact.
Buffer	A buffer in a memory area normally in the RAM. It is an area that the application has reserved for data storage.
CFG5	Configurator 5 (configuration and generation tool)
Component	CAN Driver, Network Management ... are software COMPONENTS in contrast to the expression module, which describes an ECU.
Confirmation	A service primitive defined in the ISO/OSI Reference Model (ISO 7498). With the service primitive 'confirmation' a service provider informs a service user about the result of a preceding service request of the service user. Notification by the CAN Driver on asynchronous successful transmission of a CAN message.
Electronic Control Unit	Also known as ECU. Small embedded computer system consisting of at least one CPU and corresponding periphery which is placed in one housing.
Event	An exclusive signal which is assigned to a certain extended task. An event can be used to send binary information to an extended task. The meaning of events is defined by the application. Any task can set an event for an extended task. The event can only be cleared by the task which is assigned to the event.
Interrupt	Processor-specific event which can interrupt the execution of a current program section.
Post-build	This type of configuration is possible after building the software module or the ECU software. The software may either receive parameters of its configuration during the download of the complete ECU software resulting from the linkage of the code, or it may receive its configuration file that can be downloaded to the ECU separately, avoiding a re-compilation and re-build of the ECU software modules. In order to make the post-build time re-configuration possible, the re-configurable parameters shall be stored at a known memory location of ECU storage area.
Scheduler	The algorithm which decides whether a task switch is to be affected and which triggers all necessary internal activities of the operating system is named scheduler. The scheduler decides whether a task switch is possible according to the implemented scheduling policy. The scheduler can be considered as a resource which can be occupied and released by tasks. Thus a task can reserve the scheduler to avoid task switch until the scheduler is released.
Signal	A signal is responsible for the logical transmission and reception of information depending on the restrictions of the physical layer. The definition of the signal contents is part of the database given by the vehicle manufacturer. Signals describe the significance of the individual data segments within a message. Typically bits, bytes or words are used

	for data segments but individual bit combinations are also possible. In the CAN data base, each data segment is assigned a symbolic name, a value range, a conversion formula and a physical unit, as well as a list of receiving nodes.

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CAN	Controller Area Network protocol originally defined for use as a communication network for control applications in vehicles.
Com	AUTOSAR BSW module Com
COM	Communication
DEM	AUTOSAR BSW module Diagnostic Event Manager
DET	AUTOSAR BSW module Development Error Tracer
ECU	Electronic Control Unit
EcuM	AUTOSAR BSW module ECU Manager
HIS	Hersteller Initiative Software
ID	Identifier (e.g. Identifier of a CAN message)
I-PDU	Interactive layer Protocol Data Unit
IpduM	AUTOSAR BSW module I-PDU Multiplexer
IPDUM	I-PDU Multiplexer
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR flavor)
PCI	Protocol Control Information
PDU	Protocol Data Unit
PduR	AUTOSAR BSW module Pdu-Router
RAM	Random Access Memory
ROM	Read-Only Memory
SDU	Service Data Unit
SWS	Software Specification

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com